Sewerbot + simlab: describe a job, get a machine, stress-test it, build it

A project brief for the judges. Author / team: _____ Date: _____

Contents

0. The project at a glance
1. The problem and the idea
   - 1.1 Why this is hard
   - 1.2 The idea
   - 1.3 Who it is for
2. What the user actually does (a walkthrough)
3. System overview
4. Every component, and why it exists
   - 4.1 Machines as data (the VehicleSpec)
   - 4.2 Simulation backends
   - 4.3 Tasks and classical controllers
   - 4.4 Learning
   - 4.5 The original Sewerbot: sewer inspection end to end
   - 4.6 The Builder: from real parts to a predicted machine
   - 4.7 The sewer pipe environment in MuJoCo
   - 4.8 The Goal layer: from a sentence to a checked machine
   - 4.9 Stress testing: where does it start to fail?
   - 4.10 Real hardware: the ESP32 rover
   - 4.11 The dashboard
   - 4.12 What the machines look like (and why it is not just decoration)
5. Technology choices and why
6. One example, end to end
7. Measured results
   - 7.1 Camera detector and sewer missions (original Sewerbot)
   - 7.2 The simulator itself
   - 7.3 Learning (honest)

- 7.4 The Builder against published data
- 7.5 Builder-designed machines in the simulator
- 7.6 The Goal layer and stress tests

8. Quality, testing and how the software found its own mistakes
9. Honest limits and what is not done
10. Safety, ethics and licensing
11. How to run it and a five-minute demo
12. Repository map
13. Glossary
14. Where it goes next

0. The project at a glance

In one sentence. A simulation and design platform where a person describes a job in their own words ("find cracks in a sewer pipe", "a boat that reaches people trapped by a flood, even in strong wind", "watch an area from the air"), an AI model thinks the job through (what can go wrong, what the machine needs, which real parts), and **the simulator checks every claim the AI makes**: the software shows the resulting machine doing the job in a realistic 3D simulator, tries to break it with storms, wind, mud and failures, tells the person exactly where it starts to fail and what it did *not* test, prints a build sheet, and, for one machine (an ESP32 sewer-inspection rover), drives the real hardware. Without an AI key everything still works offline from three ready-made jobs.

The problem it addresses. Building a robot today means knowing, in advance, which motors, battery, controller and sensors will work together, whether they will survive the real environment, and how to wire them safely. Beginners guess; experts spend weeks on spreadsheets and test runs; mistakes cost money, break parts, and sometimes start fires. Training or testing the robot's software is a separate, equally hard problem.

What we built, in five layers.

Layer	What it does	Where it lives
1. A simulator for any machine	Wheeled rovers, tracked crawlers, multi-rotor drones, boats (and any robot file you import) as *data*, simulated in MuJoCo; the original sewer world in Webots. The 3D view shows detailed models (a rescue boat with propeller and rudder, a rover with wheels and headlamp, a drone with spinning rotors), sky, sun and shadows, and **animated water whose waves are the same waves the physics uses**	`simlab/`, `simlab/dashboard/static/`
2. A Builder	Turns a list of real parts (57 sourced parts, 16 kinds) into predicted speed, thrust, runtime, weight; finds errors; suggests fixes that it verifies; shows the machine see-through with the chosen parts inside	`simlab/parts/`
3. A Goal tab	Free-text job → **AI planner (a free OpenRouter model) that decides, with the simulator checking it** → machine → watch it → stress test → build sheet. Offline fallback: three ready-made jobs	`simlab/llm/`, `simlab/goals/`, `simlab/hazards/`
4. Learning	Reinforcement learning for control policies, and a camera detector trained on the simulator's perfect labels	`simlab/train/`, `simlab/perception/`
5. Real hardware	An ESP32 rover and ESP32-CAM driven from the same dashboard over USB or Wi-Fi, with safety interlocks	`sewerbot/hardware/`, `firmware/`

Headline numbers (all measured; simulator unless stated). - A camera detector (RF-DETR Nano) trained purely on auto-labelled simulator images: **91.6 % / 84.5 %** precision/recall on cracks and **92.8 % / 85.8 %** on leaks, 28 ms per frame. A whole 5-stop inspection mission found **16 of 16** planted defects. - The Builder's physics matches published data (Pololu motor datasheets, a published drone thrust table): errors between **0.2 % and 13 %** on the points checked. - The Builder-designed machines then fly or float in the simulator: a 5-inch quad holds position 10/10; a hexacopter flies a waypoint course 10/10; a boat's simulated top speed is 6.16 m/s against 5.91 estimated. - A stress test of the pipe rover takes **16 seconds**; of the boat, about **2.5 minutes** (23 conditions × 4 runs); the larger four-hazard plan an AI wrote for a flood-rescue boat (25 conditions) took about 9 minutes. - Run live against a 550-billion-parameter free model (section 7.6): a flood-rescue boat plan, an underwater-cave job **correctly refused as unsupported**, and a farm-field rover whose design the Builder caught with a real wiring mistake the AI had missed. - **479 automated tests** pass; about **16,200 lines of Python** in the application, **5,400 in the tests**, and **2,700 lines of browser code**.

What is honestly *not* shown. Nothing has been proven on real pipes, real water or a real drone. The real rover's motors are being repaired, so the hardware link was tested against a software emulator of the firmware. The simulator's cracks are procedural textures, not real concrete. Every report the software writes says this itself. Section 9 lists the limits in full.

---

1. The problem and the idea

1.1 Why this is hard

1. **Parts must fit each other.** A motor rated for 6 V on a 12 V battery burns out; a controller that browns out when the motors start resets the robot; a 5 V sensor output into a 3.3 V pin can destroy the controller. These are *system* problems that no single datasheet reveals.
2. **Datasheets are optimistic.** Real motors are slower and weaker than the sheet, batteries sag under load, gearboxes lose torque.
3. **The environment is the real test.** A rover that works on a desk may stall in sludge; a boat that is fine on a calm lake may be blown away by wind; a drone that hovers may flip when one motor fails. Engineers call this *robustness*, and it is usually tested late, on the real machine, at real cost.
4. **Testing all possible scenarios is impossible**, so the honest answer is not "it is safe" but "it handles *these* conditions up to *this* level, and these others were not tested".
5. **Software and hardware are developed separately**, so control code is often first run on the real machine.

1.2 The idea

Put the *whole chain* in one tool, in the order a person thinks about it:

> **job → machine → proof it can do the job → where it breaks → how to build it → run it for real**

and make every step *checked*, *explained in plain words*, and *honest about its limits*.

1.3 Who it is for

- A **vendor or student** with a goal but no robotics background ("I need something that inspects pipes").
- A **maker** who already owns parts and wants to know if they will work together, and what to add.
- A **robotics learner** who wants a safe place to try machines, train control policies and camera detectors without risking hardware.
- A **team** that needs a first, checked draft of a design and a list of what to test next.

---

2. What the user actually does (a walkthrough)

Open the dashboard (`startup.bat`). It has six tabs: **Goal · Lab · Builder · Train · Sewer (Webots) · Hardware**. A first-time user only needs the first.

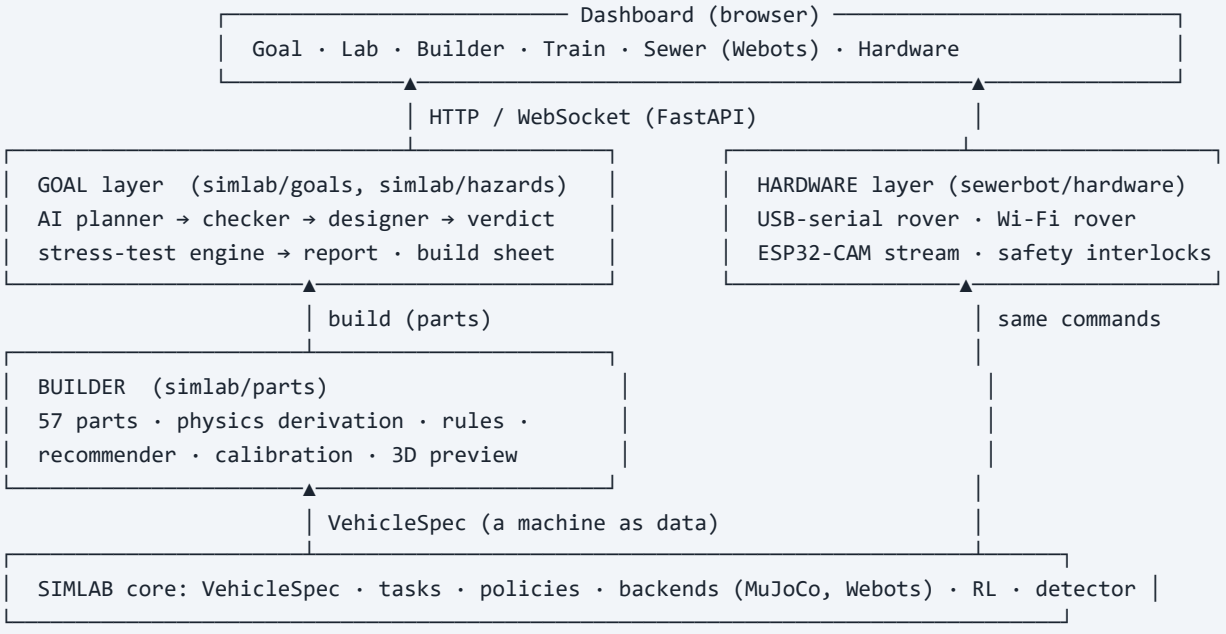
1. **Goal tab: "Describe the job".** The user types, in their own words, *a boat that goes out to people trapped by a flood, even in strong wind and rough water*, and presses **Plan it with AI**. A live panel shows what the model is doing ("Thinking through what can go wrong", "Choosing parts") and has a Stop button. A free model is slow, so this takes one to three minutes. If the job is something no machine here can do (*map an underwater cave*), the answer is "not something this simulator can do", with the reason, and **no machine is invented**. Without an AI key, or when the free quota is spent, the user opens **Ready-made jobs** (*Inspect pipes*, *Rescue on the water*, *Watch from the air*), which work offline.
2. **Only the questions that matter.** The AI asks up to three (*how far out? how much must it carry? night operation?*), each with the reason it matters and the limits the simulator supports; the user can tick **"My ESP32 rover kit"** so that parts they already own are preferred. The ready-made jobs ask three fixed questions instead.
3. **"What the AI understood" and "What could go wrong".** The page shows the AI's reading of the job, its assumptions, and every situation it thought of (strong wind, current, floating debris, a failed thruster...), each marked ✓ **can be simulated** (it will be stress-tested) or ✗ **not testable here** (it will be listed as *not tested*). A line lists what the checker **corrected or dropped** from the AI's answer.
4. **"Your machine."** The software checks the AI's designs, then searches the parts library for a verified alternative, checks every candidate, and shows the best design with a **traffic light** (green: looks right; amber: watch these; red: not there yet), three headline numbers (speed, time it can run, weight), a list of what it meets ("fits 30 cm pipes ✓", "goes 5 m and back on one battery ✓"), and the parts split into **you have** and **to buy**. If nothing in the library can do the job, it *says so* and shows the closest design with what it misses.

5. **"Watch it work."** One click opens the 3D simulator with the machine doing the job (the rover driving through a textured concrete pipe with its camera view; the boat steering to a swimmer through animated waves). For an AI plan, **"Watch in the worst conditions"** applies the waves, wind, current or grip the job asked for.
6. **"How tough is it?"** A stress test pushes the machine through the hazards the plan chose, getting worse step by step, and reports where it starts to fail (section 4.9).
7. **"Build sheet."** A printable page: the parts to buy with where every number came from and how sure we are; a wiring table; numbered assembly steps; safety rules.
8. **"Use on my real rover."** Copies the machine's size and camera mount to the Hardware tab, where the user connects the real rover and drives it with a safety checklist.
9. **"Advanced: open in the Builder."** For people who want to change any part by hand.

The same flow works for boats (a boat that goes to a person in the water), drones (an observation drone) and rovers. The simulator can only test ground rovers, drones and boats, and only the hazards in its menu (section 4.8).

---

3. System overview



The key design decision is the *seam* in the middle: a machine is described by a `VehicleSpec` (a small YAML file of physical numbers: mass, wheel radius, maximum speed, thrust, sensors...). Everything above the seam (Builder, Goal) *produces* specs; everything below it (simulation, training, the dashboard's Lab) *consumes* them. That is why a machine designed from parts runs

in every existing task and trainer with no extra work, and why new kinds of machine can be added without touching the rest.

---

4. Every component, and why it exists

For each part: **What** it is · **Why** it is there · **How** it works · **Result / limits**.

4.1 Machines as data (the `VehicleSpec`)

- **What.** A YAML description of a machine: its kind, body (mass, size), parameters, sensors, limits. Example kinds: `ground_wheeled` (2WD, 4WD, 6WD, tracked), `multirotor` (quad, hex, octo), `custom` (any MuJoCo or URDF robot file), and `boat` (added as a plug-in).
- **Why.** If machines are data, a new robot is a file, not new code, and the same task, controller and training code runs on all of them. It also lets a *form* be generated from the schema, so every parameter is validated with a clear reason when wrong (a drone that cannot hover, a boat that would sink, wheels taller than the body).
- **How.** Each kind registers a *parameter schema*, *command spaces* (how it is steered: for example `unicycle` = forward speed + turn rate; `velocity` / `position` / `motors` for drones; `helm` = throttle + rudder for boats), checks, and *derived facts* (thrust-to-weight, draft, top speed). Presets: `micro_2wd`, `mid_4wd`, `large_6wd`, `tracked_crawler`, `quad_micro`, `quad_250`, `hex_450`, `boat_small`, `boat_workboat`.
- **Plug-ins.** A new kind of machine is one Python module that registers its kind and its physics. The boat is exactly this (about 190 lines) and serves as the worked example.
- **Import.** Any MJCF/URDF robot can be imported; its actuators become raw command channels.

4.2 Simulation backends

- **MuJoCo (fast, headless, trainable).** The default engine. It runs on a CPU, is deterministic (same seed, same result), and can run many copies in parallel, which is what reinforcement learning needs. About 5,400 rover steps per second per core, 2,400 for a drone, 1,800 for a boat.
- *Ground vehicles* use a **planar model**: the body slides in x, y and yaw with acceleration limits, tyre-grip limits, and a measured yaw loss for skid-steering. Wheels are visual only. *Why:* it reproduces navigation-level behaviour at any size and speed and is fast; wheel-level contact physics is left to Webots.

- *Multirotors* are rigid bodies with per-rotor thrust (a motor lag), drag toward the wind, a battery model, and a flight controller ("autopilot") that turns commands into rotor thrusts.
- *Boats* use buoyancy sampled over the hull bottom, quadratic water drag, a thruster with lag, and a rudder.
- **Webots (visual, wheel-level physics).** The original sewer world, with textured pipes, chambers and procedurally generated crack and leak patches. One simulation at a time and slow to start, so not used for training; it is used for the realistic sewer missions and as the cross-check for the MuJoCo pipe environment.
- **A backend contract** (`Backend`) hides which engine is in use, so tasks, policies and the dashboard do not care.

4.3 Tasks and classical controllers

- **What.** A *task* defines a world, a goal, what the controller sees, and how success is scored. Built in: `goto` (reach a goal among obstacles), `waypoints` , `hover_hold` , `takeoff_waypoints` , `land` , `search` (find labelled targets among look-alikes), `sandbox` , `sewer_drive` , `pipe_inspect` (the sewer in MuJoCo), `water_rescue` (reach a person in the water).
- **Why.** Separating *what to do* from *what does it* lets any machine be tried on any suitable task, and gives training a clear reward.
- **Baselines.** Every task has a *classical controller* (hand-written). **Why this matters:** it is the yardstick. A learned policy is only worth reporting if it beats the classical one. The measured result (section 7) is honest: it did not.

4.4 Learning

- **Reinforcement learning (Train tab).** Stable-Baselines3 (PPO or SAC) with 1–16 parallel simulators, an optional easy-to-hard curriculum, live learning curves, and a final comparison with the classical controller on identical unseen scenarios (with 95 % confidence intervals). *Why:* to let someone train a control policy for their own machine and see, truthfully, whether it is better than the simple one.
- **Camera detector (Train tab and `tools/`).** The simulator renders a second, *segmentation* image where every object has a unique colour, so bounding boxes are **pixel-exact and occlusion-aware at zero labelling cost**. A pipeline captures views, trains an **RF-DETR Nano** detector (Apache-2.0 licence, chosen to avoid licence traps), and evaluates it. *Why:* labelled data is the expensive part of computer vision; here the simulator labels itself.
- **Hard negatives.** The images deliberately include things that must *not* be flagged (pipe-joint rings, brick chambers, silt, goal posts), which is what makes the detector usable.

4.5 The original Sewerbot: sewer inspection end to end

This is the first project the platform grew from.

- **Sewer maps (YAML).** Pipes (round, oval, box), chambers, branches, goal posts and planted or random cracks and leaks. Four maps: `short_pipe`, `straight_t`, `star_five_goals`, `large_loop`.
- **World generator** (`tools/gen_world.py`). Turns a map and a rover profile into a Webots world with procedural wall textures and baked defect patches. **Collision uses simple boxes** (one thin box per wall segment) because Webots' physics was unstable on concave meshes; meshes are used only for appearance.
- **Rover profiles** (`micro_2wd`, `mid_4wd`, `tracked_crawler`, `large_6wd`): size, drive, speed, camera height/tilt/field of view. Skid-steer rovers turn less than the ideal maths says, so each preset stores a **measured yaw gain** (0.87 for 2WD, 0.67 for 4WD and tracked, 0.66 for 6WD).
- **Detectors.** RF-DETR (learned) and a training-free OpenCV fallback ("classical"). The fallback is kept deliberately *as a baseline*: on the same frames it scored only 1–3 % precision, which shows what the learned detector buys.
- **Defect tracker.** A crack seen in dozens of frames must become *one* defect on the map. The tracker projects each detection onto the pipe wall, merges duplicates, requires several sightings, and ignores low-confidence boxes. Tightening it cut false alarms from 21 to 6 at the slowest speed.
- **Mission and report.** The rover drives a route (manual, go to a goal, or inspect everything), the tracker builds a defect map, and a self-contained HTML/JSON report is written (map, defects with snapshots, and a score against the planted ground truth).
- **Camera realism.** Real cameras auto-expose and a rover's lamp lights only part of the pipe, so the simulator emulates **auto-exposure** and tilts the camera *up* about 10° (a level camera on a 7.5 cm rover sees mostly floor). Without this, labelled defects are invisible and the detector learns noise.

4.6 The Builder: from real parts to a predicted machine

Why it exists. A user has an ESP32, a motor driver, motors, wheels and a battery; they need to know what that machine can do and whether it will work. Nothing connects the parts on the bench to the numbers a simulator needs.

- **Parts library** (`simlab/parts/library/<kind>/*.yaml`): **57 parts in 16 kinds**: DC motors, wheels/tracks, motor drivers, batteries, voltage regulators, controller boards, cameras, range sensors, IMU, GPS, lamps, drone frames, motor+propeller sets, ESCs, boat hulls, boat thrusters.

- Every part carries a `source` (where the numbers came from: a datasheet link, a test report, or "generic") and a `confidence` (`datasheet`, `typical`, `assumed`, `measured`) shown as a badge. *Why:* hobby parts are often mis-specified; the user must be able to see which numbers to distrust.
- Users can **add their own parts**. The form is generated from the same schema the server checks, and a unit mistake is refused with a reason (a motor whose numbers imply more than 90 % efficiency is almost always torque typed in the wrong unit).
- **Physics derivation** (closed-form, no hidden tuning knobs):
- *Ground:* a brushed DC motor has a straight torque-speed line fixed by four datasheet numbers (rated voltage, no-load speed, stall torque, stall current). Fed from a battery that sags and through a driver that loses some voltage, the maths gives top speed (against rolling resistance and drag), acceleration, climbing slope, turn rate, stall current and runtime. Assumptions are listed on screen: a 0.85 torque derate, 80 % usable battery capacity, 10 % extra mass for wiring.
- *Air:* thrust and current scale with battery voltage squared from a published thrust-test point; hover current follows a curve fitted to measured data; the autopilot's limits (tilt, climb rate) come from rules of thumb and are conservative.
- *Water:* draft, freeboard and top speed from the boat plug-in's own model.
- **Rules and findings.** Every problem is a *finding* with a severity (error blocks saving, warn, info), a plain explanation and often a fix. Examples: the **documented ESP32 brown-out on 4×AA NiMH without a boost converter**; a battery above the motor's or driver's rating; a stalled motor tripping the driver; a 5 V sensor into a 3.3 V pin; not enough controller pins; a drone with thrust-to-weight below 1.3 ("it will not fly"); a boat that sinks.
- **Verified suggestions.** For a finding with a fix ("add a boost converter"), each candidate part from the library is *applied to a copy of the build and the whole build re-checked*; a part is offered only if it removes the problem without creating a new error. Parts the user owns come first. *Why:* a suggestion that breaks something else is worse than none.
- **Task fit.** A task states what it needs (a camera, a lamp in a dark pipe, that the machine fits the pipes, enough runtime); the Builder checks the build against it.
- **Calibration.** The user enters what they can measure with a ruler, a stopwatch and a kitchen scale (mass, distance in 3 s, spin angle, time to 90 % speed, runtime); the model is scaled to match, and a table shows predicted versus measured. *Why:* it is the only honest way to tie an estimate to a real machine.
- **3D preview.** The assembled machine is drawn with its parts inside a see-through body, and each sensor's field of view as a cone.

4.7 The sewer pipe environment in MuJoCo

- **Why.** Webots is realistic but slow and cannot run in parallel, so training and quick tests could not use the pipes. This environment brings the sewer into the fast engine.
- **How.** The same sewer maps are turned into textured meshes, collision boxes **identical to Webots' (a test compares them)**, labelled defect patches (so segmentation labels work), the rover's own **lamp as the only light**, and the same camera auto-exposure. A technical detail worth knowing: MuJoCo lights each vertex, so a big wall shows black unless it is subdivided into cells under 0.45 m; the first attempt produced 102,000 slivered triangles, the grid approach about 10,000.
- **Result.** About 1,790 policy steps/s without a camera and 22 with a 640×480 frame every step; an inspect-everything mission scores like the Webots one.

4.8 The Goal layer: from a sentence to a checked machine

- **The AI planner** (`simlab/llm/`). The person's sentence goes to a language model (OpenRouter's free tier; the default is NVIDIA Nemotron 3 Ultra, 550 billion parameters) with a prompt **built from the live parts library, hazard menu and task presets**, so the model can never be told about a part or hazard that does not exist. It returns one JSON plan: what the job is and which machine type fits (or "unsupported"), the job's numbers, at most three questions, 4 to 8 scenarios it thought of, extra requirements, which test scene and which hazards to sweep, and one or two complete designs. *Why a model at all:* three fixed jobs and keyword matching could not understand "people trapped by a flood"; a model can read any job and brainstorm what can go wrong.
- **The principle: the AI proposes, the simulator checks** (`schema.py`, `metrics.py`, `design.py`, `synth.py`). The model's answer is *data, never instructions*, and is checked before anything happens:
- **Closed menu.** Parts, hazards, faults, task presets and job numbers come from a menu built from the code. Anything the model invents is dropped and listed ("dropped requirement ...: not something the Builder reports").
- **Clamped to physics.** Every number is held to the range the simulator supports (a model asked for 100 to 150 kg of payload where the boat model carries 20: it was clamped and the page said so). Even the *default* of a question is clamped, and the limit is shown beside the box.
- **Measured, not believed.** A requirement such as "freeboard of at least 25 cm" is computed by the Builder's evaluator; a design the AI claims is fine but whose battery or wiring is wrong is flagged by the same electrical checks as any hand-made design. The live run in section 7.6 shows this catching a 5 V sensor wired to a 3.3 V pin that the model missed.
- **A scenario the simulator cannot model is never passed.** The AI gives each scenario a hazard id or `null`; `null` ones are shown as "not testable here" and end up in the report's *not tested* list.

- **Untrusted input.** The job text cannot widen what the model may choose: an instruction hidden in the sentence changes nothing, because the model's output goes through the same validation.
- **At most three requests per plan** (one to plan, one repair if the answer was unreadable or cut off, one revision when the checker finds problems), because the free tier allows about 50 requests a day. When the AI fails or no key is set, the **offline planner** (below) runs and the page says so.
- **The key stays on the machine.** It is stored in a git-ignored file or read from `OPENROUTER_API_KEY`, never sent to the browser, never written to a log or an error message.
- **Goal catalogue and offline planner** (`simlab/goals/catalogue/*.yaml`): *Inspect pipes*, *Rescue on the water*, *Watch from the air* (observation only). Each has keywords, up to three plain questions whose answers are numbers, and the task to test on. Matching is deterministic keyword scoring that works with no internet; no match → ask, never guess. This is the fallback, and the path used when no AI key is set.
- **Designer** (`design.py`). The AI's own designs are checked first; then a *staged search* over the verified library finds alternatives. Every candidate (about 210 for the rover, 288 for the drone, 12 for the boat, 2–4 seconds) must (1) be a valid build per the Builder's checks and (2) meet the goal's requirements, which the answers (or the AI's job numbers) turn into numbers:
- *Pipes:* fits the pipe width; drives the distance and back with a 1.5× margin; enough pulling power for wet pipes.
- *Water:* stays out the time; round trip within 80 % of the runtime; freeboard (hull above the water) at least 0.7× the wave height; thruster at least 1.3× the wind's push on the hull; a second thruster for rough water (the rescue-boat rules ask for two independent propulsion systems offshore).
- *Air:* flight time 1.2× the request; thrust-to-weight enough to lean into the wind. It prefers parts the user owns, then fewest warnings, then lowest weight, and offers extra designs only when they are clearly different (15 % longer time out or 15 % lighter). Finalists are re-checked against the goal's own task. **If nothing qualifies it says so and shows the closest design and what it misses.**
- **Plain-language verdict.** Findings already carry plain titles, so the screen shows "A stalled motor would overheat the driver" instead of a rule code, with a traffic light.
- **Build sheet** (`assembly.py`). Parts list (quantity, have/buy, confidence, source), wiring, steps, safety rules, printable. **Pin-level wiring and flashing steps exist only for the ESP32 + DRV8833 two-motor rover**, because that is the one machine whose firmware and wiring were built and checked; the sheet for a drone or boat says to wire from the datasheets. *Why this restraint:* a wrong wire on a LiPo battery can start a fire, so the tool claims only what it has verified.

4.9 Stress testing: where does it start to fail?

- **Why.** "Handles every scenario" is impossible to prove and dangerous to claim. What *can* be done honestly is to push the machine through conditions that get worse step by step and report **the level at which it starts to fail**, together with a list of what was not tested. This follows how safety engineers describe scenario-based testing (define the operating conditions, sweep them, report coverage and what is outside).
- **Hazard hooks.** `Sim(vehicle, task, hazards={...})` applies fixed, repeatable conditions to each kind of machine:
- *Rover:* grip, mud/debris (a share of the motors' pulling force, which cuts top speed by the same share: a linear motor curve), slow or lost commands, a dead drive motor, battery level.
- *Boat:* waves (a few linear deep-water waves with a realistic spread, sampled over the hull), wind (quadratic drag on the part above water, which also heels the boat), current, thruster failure, a jammed rudder, battery; the boat reports roll, pitch and "deck awash" so capsizing and swamping are detected.
- *Drone:* wind with gusts and quadratic drag on the airframe, rotor failure, weak motors, battery.
- **Same scenes at every level.** The wave phases, wind and current directions come from the seed, not the severity, so the only thing that changes between levels is the hazard.
- **Catalogue** (`simlab/hazards/catalogue/*.yaml`). Each hazard lists its levels, a plain explanation, a **source** for the range (for example the operating categories of rescue boats, published wind ratings of search-and-rescue drones, reports of what stops sewer robots), and a suggested fix.
- **Result.** For each hazard: the last level that passed and the first that failed, compared with what the user asked ("you asked for 8 m/s wind: covered"). Faults (a thruster, rotor or motor failing) are single cases. A **weakest point** is named with a suggested fix. It runs in the background with a progress bar and a Stop button, and writes an HTML and JSON report.
- **What it never says.** That the machine is safe. Every report ends with a **"not tested"** list (for example breaking waves and hull slamming, darkness and a fogged lens, a blocked pipe, GPS loss, a drifting person) and a statement that results come from a simple built-in controller in a simulator.
- **Why the stress test matters for the software too.** It found three of our own mistakes (section 8).

4.10 Real hardware: the ESP32 rover

- **The machine.** An ESP32 DevKit (brain), a DRV8833 motor driver on GPIO 26/27 (left) and 32/33 (right), two TT-class gear motors in a ZK-2WD chassis, 4×AA NiMH, and a **separate ESP32-CAM** board streaming video (GC2145 sensor, 96×96 pixels, about 9–11 frames per second). Planned sensors: an HC-SR04 ultrasonic sensor and an IR obstacle sensor.

- **Why two control paths exist.** The original design (`firmware/esp32_rover`) is a Wi-Fi rover (UDP drive commands, a 0.3 s failsafe, a battery cut-off, a camera stream). The rover actually built uses a **USB cable** and its own simple firmware (`rover_wasd` : single keys `w a s d x e q`). The dashboard supports both.
- **The USB driver** (`sewerbot/hardware/serial_rover.py`). Speaks the rover's key protocol, and adds safety the firmware cannot give alone:
- The first byte after boot is a **stop**; a **heartbeat** is sent every 0.2 s; a drive command **lapses after 0.4 s** unless renewed; stop is the first thing done when closing.
- Forward motion is **refused** when the distance sensor reports an obstacle under 15 cm, gives no valid reading, or has gone silent, or when the IR sensor sees something. (An unplugged sensor must never look like a clear path.) Spinning remains allowed so the rover can turn away.
- Opening a COM port resets the ESP32, so the driver waits 2.5 s; only one program can hold a port, so a busy port gives a message telling the user what to close.
- **The new firmware** (`firmware/esp32_rover_serial`). The user's sketch plus: a telemetry line every 0.5 s, optional ultrasonic and IR sensors that stop forward motion on their own, and a **watchdog that is always armed while the rover moves** (the original only armed after the first heartbeat, so a serial monitor could leave it running). It **compiles** for the ESP32 DevKit (21 % of flash, with or without the sensors). **It has not been flashed.**
- **The pre-drive checklist.** Drive controls stay locked until four boxes are ticked (wheels off the ground or the area clear; battery at least 4.4 V; Stop tested; power switch within reach). The lock is enforced **on the server**, not only in the browser. **Stop always works.**
- **The emulator.** A software copy of the firmware (same keys, replies, watchdog and sensor behaviour) lets every part be tested and demonstrated without the rover. *Why:* hardware is slow, fragile and unavailable at 3 a.m.
- **Calibration on hardware.** Drive for a fixed time, measure the distance and the turn, enter them; speed and turn rate are stored for the current speed level.

4.11 The dashboard

A FastAPI server and a plain-JavaScript page (no build step), with three.js for the 3D view. **Why plain JavaScript:** nothing to install or compile, easy to read. All tab scripts share one global scope, so a test audits that no two files declare the same top-level name (this bug broke the Builder twice).

Tab	Purpose
Goal	The starting point: describe a job → AI plan (live progress, questions, what could go wrong) → checked machine → stress test → build sheet; AI settings box (key, model, how many free requests are left); ready-made jobs as the offline path
Lab	Pick any machine and task and watch it in 3D with its camera and telemetry; drive it by hand; a form to edit every parameter; **Conditions** sliders for waves, wind, current and grip
Builder	Assemble a machine from parts; predicted performance, findings, suggestions, calibration, a see-through 3D preview with the parts inside
Train	Reinforcement-learning and detector training runs, live curves, comparison with the classical controller
Sewer (Webots)	The original sewer inspection missions with the detector and a mission report
Hardware	Connect and drive the real rover and camera, checklist, calibration, save frames, replay video, check the firmware compiles

4.12 What the machines look like (and why it is not just decoration)

The first version drew every machine as plain boxes on a flat plane ("two blocks running in the water"), which made results hard to believe and hard to read. The 3D view (`simlab/dashboard/static/`, three.js) now draws:

- **Models built from the machine's real dimensions** (`models/boat.js`, `rover.js`, `drone.js`, `props.js`): a rescue boat (shaped orange hull, console, mast with light, lifebuoy, outboard with a spinning propeller and a rudder that turns), a rover (body shell, wheels or tracks that turn with the real wheel motion, camera on a stalk, headlamp), a quadcopter or hexacopter (frame, arms, motors, propellers that spin with thrust), a swimmer in a life jacket, floating debris. They are procedural (no downloaded art), so the project stays offline and licence-clean; they are *typical designs*, not CAD of a product.
- **A sky, sun and shadows, and animated water whose waves come from the same five-wave model the physics uses** (`env.js`; the shader and `BoatPlant.surface` use the same heights, directions and clock). The boat visibly rides the wave the buoyancy calculation is using. In pipes the world is dark and lit by the vehicle's own lamp, on textured concrete and brick with solid crack and leak patches.
- **A see-through "ghost" in the Builder**, with the battery, controller, motors or thrusters, sensors and each camera's field of view drawn inside the same model.

- **For the onboard camera** (rendered by MuJoCo, separate from the browser view) open water has a textured, glinting sea under a pale sky. Other scenes are unchanged on purpose, so previously trained detectors see the same images. Fog was tried and rejected: it hid the person the boat is looking for.
- **Cost.** The heaviest scene (boat in rough water) draws with 53 draw calls and about 36,000 triangles and keeps the display's full frame rate.

---

5. Technology choices and why

Technology	Used for	Why this and not something else
Python 3.11	Everything except firmware and browser code	The robotics, ML and web ecosystems are here; one language across simulator, ML and server
MuJoCo 3.17	Fast physics	Free, deterministic, very fast on CPU, parallelisable: the properties RL and sweeps need
Webots R2025a	Realistic sewer world	Apache-2.0, strong wheel-level physics and rendering; kept for fidelity, not speed
Stable-Baselines3 (PPO, SAC)	RL	Standard, well-tested implementations; honest comparison with baselines
RF-DETR Nano	Camera detector	Apache-2.0 (no licence strings), accurate, 28 ms per frame; the AGPL YOLO family was deliberately *not* installed
OpenCV	Image handling, classical detector	Standard; the training-free baseline
PyTorch (CUDA 12.8)	Detector training on an RTX 5050 8 GB laptop GPU	The GPU class here needs CUDA ≥ 12.8; small model chosen for the 8 GB limit
FastAPI + WebSockets	Server	Typed endpoints, live state streaming, tiny code
Plain JS + three.js	Browser UI and 3D	No build tooling; the viewer draws any scene
OpenRouter (free models)	The AI planner	One API to many models, a free tier, and the **model can be swapped by typing a name**; the project does not depend on one vendor. The default is NVIDIA Nemotron 3 Ultra (550 B). Optional: without a key the offline planner is used
YAML	Machines, parts, tasks, goals, hazards	Human-readable and editable; the *data* is separated from the *code*
ESP32 DevKit + ESP32-CAM + DRV8833	The real rover	Cheap, available, Wi-Fi/USB, enough pins; a camera board cannot run a useful detector, so the laptop does the detection

Technology	Used for	Why this and not something else
Arduino-ESP32 core 3.x, arduino-cli	Firmware	Compile checks can run from the dashboard
pyserial	USB link	The standard way to talk to a COM port
pytest	479 tests	Every safety rule and every claim in this brief is backed by a test or a measured run

---

6. One example, end to end

"*Find cracks in a sewer pipe*", *30 cm pipes, one street block, wet inside, using my rover kit.*

1. The matcher picks *Inspect pipes*; the answers become `pipe_d = 0.30`, `distance = 50 m`, `wet = 1`.
2. The designer builds the rover's chassis to fit the pipe (width about 13 cm), then tries 210 combinations of motor, wheel, driver and battery. It marks the TT motors, 65 mm wheels, DRV8833 and ESP32 camera board you own as "have", and the boost converter, ultrasonic sensor and lamp as "to buy".
3. The verdict: "*Should do the job, with 5 things to watch*" (for example, a stalled motor would overheat the small driver; the ultrasonic echo needs a voltage divider; the 96×96 camera is far below what the task needs).
4. *Watch it work* drives the machine through a simulated pipe.
5. The stress test (16 s): it is fine until mud eats 60 % of the pulling force, and **fails every run if a drive motor dies** (a 2-wheel rover circles). The weakest point is named, with the suggestion "a drive layout with more motors keeps some drive on both sides". The "not tested" list includes blocked pipes, darkness and real cracks.
6. The build sheet lists the 9 parts with sources, the GPIO-to-driver pin table, and steps that start with "set the boost converter to 5.1 V before connecting any board".
7. *Use on my real rover* copies the size and camera mount to the Hardware tab; the user connects `COM6`, ticks the checklist and drives with the live camera view.

---

7. Measured results

All numbers are measured on this project's machine (Intel Core 7 240H, RTX 5050 8 GB, Windows 11). **Simulator data unless stated.**

7.1 Camera detector and sewer missions (original Sewerbot)

RF-DETR Nano, 25 epochs, 55 minutes, dataset `sewer_v1` (4,300 train / 650 valid / 450 test images from 3 maps × 3 rovers × 4 random-defect seeds, about half with no defect):

Class	Precision	Recall
Crack	91.6 %	84.5 %
Leak	92.8 %	85.8 %

Validation mAP@50:95 = 0.70; 28 ms per frame. The training-free OpenCV baseline scored 1–3 % precision on the same frames. **Whole missions:** `star_five_goals`, all 5 goal posts visited, **16 of 16** planted defects found (5 false alarms, 76 % precision, before a later refactor). After the refactor the same mission still found 16/16 but with **9 false alarms (64 %)**; both fast and real-time modes agree and **we did not find why**. Speed sweep with 60 ms motion blur: recall stayed at 12 of 14 from 0.2 to 1.5 m/s; precision rose from 67 % to 100 % as speed rose (slow runs give a stray detection more chances to be confirmed).

7.2 The simulator itself

- Rover top speed 0.500 m/s against 0.5 specified; acceleration 0.599 against 0.6; turn rate 1.500 against 1.5 rad/s; no sideways drift.
- Drone hovers within 3 cm and 1° on a 45 g quad, a 650 g quad and a 1.8 kg hex; a steady 6 m/s wind is rejected to under 0.15 m/s.
- Boat floats at its predicted draft (±1 cm); top speed 5.04 m/s against 4.9 estimated.
- Speed: 5,400 rover policy steps per second per core; 3,900 per second across 8 training processes; onboard camera 13 ms (320×240) to 21 ms (640×480).

7.3 Learning (honest)

Classical baselines: rover `goto` 154/160 goals reached with zero contact; drones 150/150; `search` 18/18 for the faster machines. PPO (8 simulators): the rover policy reached the goal 98 % of the time but **scraped obstacles (21 contact steps per episode)** where the baseline

touched nothing; doubling the training and tripling the collision penalty made it *worse* (93 %, 62 contact steps). The drone policy matched the baseline (100 %) but was slightly slower. **Conclusion stated in the repository: the learned policies match the classical controllers; they are not better.** A camera detector (RF-DETR) trained on drone views found crates, barrels and balls with 94.7–97.6 % precision and 100 % recall (an easy problem: distinct solid colours).

7.4 The Builder against published data

Check	Published	Model	Error
Pololu HP motors, maximum output power (3 motors)	1.3 / 1.4 / 1.3 W	torque × speed / 4	+0.2 / −2.3 / +4.1 %
Pololu maximum-efficiency point	490 rpm, 0.32 A	487 rpm, 0.36 A	−0.6 % speed, +12.5 % current
Pololu Zumo, two gear ratios (wheel radius fitted on the third)	measured speeds	predicted	−7.5 % and −1.2 %
BrotherHobby 2306 motor + 5×4 prop on 4S: current at five thrusts	1.3 / 2.7 / 8.0 / 17.2 / 31.1 A	fitted curve	+11.1 / −9.1 / −7.2 / −1.1 / 0 %

(The curve is fitted at its top point, so the 0 % there is by construction.) A calibrated vehicle then reproduces the measured top speed within 4 %, a spin within 12 %, and time-to-90 % within about a third. One counter-example is kept on purpose: a common TT motor's datasheet says 200 rpm while a measurement gave 250, which is why every part shows its confidence.

7.5 Builder-designed machines in the simulator

5-inch quad `hover_hold` 10/10; F550-class hexacopter `waypoints_easy` 10/10 (thrust-to-weight 1.33); boat top speed 6.16 m/s against 5.91 estimated. A scenario sweep of about 30 builds across ground, air and water behaved as expected: each broken build is refused for the right reason (no boost converter → brown-out error; a 2.5 kg payload on a small frame → "will not fly"; ballast beyond displacement → "sinks").

7.6 The Goal layer and stress tests

- Designer speed: 2–4 seconds per goal.
- Pipe rover stress test (4 runs per level, 16 s): mud passes to 30 % of the pulling force and fails from 60 %; a dead drive motor fails every run.

- 1.2 m boat (23 conditions, about 2.5 minutes): wind passes to 12 m/s and fails at 16; losing the only thruster fails every run; a jammed rudder 25 % success; fixed obstacles: 6 or 15 in the way give 50 %, 30 give 0 %.
- Drone: wind passes to 8 m/s, 50 % at 12, fails at 16 and 20; **one failed rotor flips it every time** (the simple autopilot has no fault-tolerant control).

The AI planner against a real model (2 November 2026, NVIDIA Nemotron 3 Ultra 550 B through OpenRouter's free tier; each plan stage used 1 to 2 of the 50 daily requests and took about 1.5 to 3.5 minutes):

Job typed	What happened
"A boat that goes out to people trapped by a flood and brings help, even in strong wind and rough water."	Asked three questions (range, load, night operation), then wrote a plan with 6 scenarios (5 simulatable: waves, wind, current, debris, thruster failure, combined storm; 1 not testable). **No design in the library met every requirement**, and the page said so ("Closest we found", red): 24 cm of freeboard against about 42 cm advised for 0.6 m waves, and a thrust-to-weight ratio of 1.5 that the model had inferred as a requirement but the best design reached only 0.30 of (the model's inference may itself have been too demanding for a boat; the page labels it "the AI's inference"). The 25-condition stress test (about 9 minutes) found waves and wind and current handled up to the asked levels, **fixed obstacles the weakest point (fails from 6 in the way, 15 asked: not covered)** and a jammed rudder failing every run
"Map an underwater cave system, going 30 metres deep, and bring back 3D scans."	Answered **unsupported** in one request, with the reason (needs a pressure-rated submersible, none of the three machine types). No machine was invented
"A small rover that drives along the rows of my vegetable field after rain and checks the plants for disease with a camera."	A ground rover design that **met every requirement but was marked amber**: the Builder's electrical check found the ultrasonic sensor's 5 V output wired to a 3.3 V pin, which the model had missed ("add a level shifter"). The simulator's ground menu only has "pipe width", so the field rover was checked as if it had to fit a 50 cm pipe

These runs also showed three faults in the project's own code, now fixed (section 8).

---

8. Quality, testing and how the software found its own mistakes

- **479 automated tests** (27 test files, no Webots or hardware needed; the AI planner is tested against a local fake of OpenRouter, so no key, internet or quota is used): physics identities (a motor at rated voltage unloaded turns at rated rpm), monotonicity (more mass is slower; more cells is faster), electrical rules, the API, safety interlocks, the hazard hooks, report contents (a test checks that no report contains the word "safe" and that every report lists what was not tested), and the Webots backend against a fake controller.
- **Honesty rules enforced in code:** a report always has a "not tested" list; the build sheet only generates wiring for the verified rover; the designer says "nothing in the library meets this" instead of faking a pass; every part shows its source and confidence.
- **Mistakes the project's own tools caught:** (1) the stress test showed the first rescue-boat controller failed even in calm water (a heavy boat approached too fast to stop), so it now brakes according to its own thrust-to-weight; (2) the stress test showed the designer's "best" 3 m boat was blown away by 8 m/s wind, so a wind-versus-thrust requirement was added; (3) the mud hazard first barely changed anything because the traction model is stiff, so it now cuts speed by the share of force used up. (4) Running the AI planner for real found three faults that the fake server had hidden: OpenRouter refuses the Inkling models to ordinary API clients with a 403 ("only for agentic harnesses"), which the software mislabelled as "bad key" and which stopped its fallback chain (now: the model is skipped, remembered for six hours so it does not cost a request again, and the next model is used); a reasoning model spends its output budget thinking, so a 3,500-token limit cut the plan off mid-answer (now 8,000, and a cut-off answer is asked again, shorter, and recorded on screen); and a thinking model may print a small example object before its real answer (the reader now takes the largest complete object, not the first). (5) The ready-made "Rescue on the water" job pointed at an idle sandbox, so "Watch it work" never showed a rescue; it now runs the real rescue task. Earlier: two library entries with wrong numbers (a 75C battery pack giving 112 A; a frame envelope) were caught by sanity tests, a Windows networking bug (two programs sharing one port, so the rover's first message vanished) was found and fixed, and a leaked port from a test was caught by the full run.
- **Reproducibility:** every run is seeded; the hazard levels share scenes; results in this document come from scripts in the repository.

9. Honest limits and what is not done

1. **No real-world proof.** Nothing here has been shown on real pipes, real water or a real drone. The simulator's cracks and leaks are procedural textures; a detector trained on them will not read real concrete reliably. A path to fix this exists (capture frames from the real camera, label them, fine-tune; `docs/TRAINING.md`).
2. **The real rover is not yet running.** Its motors currently hum without turning (a hardware problem being repaired). The USB link and the new firmware were verified against a software emulator and by compiling; **the new firmware has not been flashed**.
3. **The real camera is too small for detection.** It streams 96×96 pixels; real cracks cannot be resolved. In the real-rover flow the camera is for driving and for saving frames; detection is demonstrated in the simulator.
4. **Estimates are optimistic until calibrated.** The Builder's predictions come from datasheets; expect tens of percent error until the user enters measurements (calibration exists for ground vehicles only; air and water are first guesses).
5. **Learned control policies are not better than the classical ones** (section 7.3), and none has been transferred to hardware.
6. **The boat wave model is simple.** Waves act through buoyancy only (no breaking waves, slamming, diffraction or horizontal wave force), so "passes at 0.9 m waves" is the easy half of the question. Fixed obstacles do not move when hit.
7. **Stress tests use a simple built-in controller**, and test only what is listed. A better controller may do better; hazards not modelled are not tested.
8. **An unexplained difference:** the Webots star-map mission gives 9 false alarms against 5 before a refactor, and the cause is unknown.
9. **The AI planner is only as good as a free model.** It is slow (1.5 to 3.5 minutes per stage), the free tier allows about 50 requests a day shared by every free model (so roughly 15 to 25 plans a day), and free endpoints come and go. It ignores limits it is told (it asked for 100 to 150 kg where the simulator carries 20) and infers requirements that may be unreasonable (a thrust ratio of 1.5 for a boat). The checker catches the first kind and labels the second ("the AI's inference"), but a plan is a *proposal*, not engineering advice. The three designs of three live runs are not a statistical sample.
10. **The AI can only choose among what the simulator can test.** It will think of hazards that are not modelled (breaking waves, corrosion, rain on a lens, uneven ground); these appear as "not testable here" and in the *not tested* list, never as a pass. A job outside the menu is stretched, not refused: the ground menu only knows pipe width, distance and wet floors, so a field rover is checked as if it had to fit a 50 cm pipe. The planner cannot design a machine the parts library lacks.

11. **Visuals are typical designs, not CAD.** The 3D models are procedural and look like a rescue boat, a pipe rover and a quadcopter; they are not a particular product. Waves in the 3D view match the physics for the boat only; the drone's and rover's physics are unchanged.
12. **Not built:** prices and availability, a drifting person in the rescue task, firmware generation, driving real drones or boats (the hardware link exists only for the ESP32 rover), fixed-wing aircraft, a motor thermal model, autonomous driving on the real rover (it has no encoders or IMU to know where it is).

---

10. Safety, ethics and licensing

- **Physical safety.** The rover interlock (checklist, stop always available, heartbeats, obstacle refusal, watchdogs on both sides) exists because a wrong command to a real motor can injure someone or break the machine. Build sheets carry specific warnings (battery polarity, setting the boost converter before connecting boards, propellers off for bench tests, LiPo handling, sealing electronics on a boat, a kill switch).
- **No safety claims.** The software never says a machine is "safe"; it says what was tested, to what level, and what was not.
- **Use of the drone goal.** The drone goal is **observation only** (search and watch). The platform does not design, test or support intercepting or disabling other aircraft.
- **Licences.** Webots and RF-DETR Nano/Small/Medium/Large are Apache-2.0; OpenCV, NumPy, FastAPI and PyTorch are BSD/MIT-style. The AGPL Ultralytics YOLO family is deliberately **not** installed (using it would require open-sourcing the whole project). No public datasets were used: textures and defects are generated by this repository. See `docs/LICENSING.md`.
- **Data.** Everything runs on the user's computer, with **one optional exception**: the AI planner. If (and only if) an OpenRouter key is set, the job sentence, the user's answers to the AI's questions, and the public parts list and menu are sent to OpenRouter and the model provider. **Free models may keep what is sent**, so a job description should not contain anything confidential; the settings box and `docs/PLANNER.md` say so. Nothing else is sent: not the machine designs the user saves, not the hardware, not the camera frames, not the key itself beyond the request header. With no key set (the default), nothing leaves the machine and the offline planner is used. The key is stored in a git-ignored file or an environment variable and is never shown again in the browser.
- **Model terms.** Some free models are open only to approved apps: OpenRouter refuses the Inkling models to ordinary API clients, and this project does not disguise itself as one of those apps. It uses a model that serves ordinary clients instead.

11. How to run it and a five-minute demo

Run: double-click `startup.bat` (or `.venv\Scripts\python.exe -m sewerbot`), open `http://127.0.0.1:8000`. Tests: `.venv\Scripts\python.exe -m pytest -q`. Setup notes: `docs/SETUP.md`.

A five-minute demonstration 1. **Goal tab, offline path** (1 min): open *Ready-made jobs*, choose *Inspect pipes*, tick *My ESP32 rover kit*, press *Design my machine*. Point out the traffic light, what it meets, and "you have / to buy". 2. **Watch it work** (30 s): the yellow rover drives through the textured concrete pipe with its camera view. 3. **How tough is it?** (1 min, 16 s of compute): show where it fails and the "not tested" list. Open the full report. 4. **Build sheet** (30 s): parts with sources, the pin table, safety box. 5. **The AI planner** (needs a key and 2 to 4 minutes, so start it first and talk while it runs): describe a flood-rescue boat in your own words; show the live phases, the questions, **"What could go wrong"** with the ✓ / ✗ marks, then the honest red "Closest we found". Then type an underwater-cave job to show the **refusal**. 6. **Watch in the worst conditions** (30 s): the orange boat on animated 0.6 m waves in 15 m/s wind. 7. **Hardware tab** (1 min): press *Try USB emulator*, show the checklist locking the controls, unlock, drive, show the obstacle-refusal behaviour.

12. Repository map

Path	What is in it
`simlab/spec`, `vehicles`, `registry.py`	Machines as data: schemas, kinds, presets
`simlab/backends`	MuJoCo (plants, XML, sensors, rendering) and Webots adapter
`simlab/tasks`, `control`	Tasks and their classical controllers
`simlab/train`, `perception`	Reinforcement learning; detector capture/training/evaluation
`simlab/parts`	The Builder: library, schema, derivation, rules, recommender, calibration, preview
`simlab/llm`	The AI planner: OpenRouter client (streaming, retries, model fallback), settings and the private key file, the menu, prompt builder, plan validation and clamping, requirement metrics, the planning flow
`simlab/goals`, `simlab/hazards`	Goal catalogue, matcher, designer, build sheet; hazard catalogues, sweep engine, reports, and `synth.py` that turns an AI stress plan into a sweep
`simlab/dashboard/static/models`	The 3D vehicle models (boat, rover, drone, people, debris, pipe materials)
`simlab/plugins`	The boat plug-in and its presets
`simlab/world`	Pipe geometry for the MuJoCo pipe environment
`simlab/dashboard`, `sewerbot/dashboard`	The server and the browser pages
`sewerbot/core`, `brain`	Sewer geometry, rover profiles, detectors, tracker, mission, reports
`sewerbot/hardware`	The Wi-Fi and USB rover drivers, camera client, emulators
`firmware/`	`esp32_rover` (Wi-Fi), `esp32_rover_serial` (USB), `esp32cam_stream`, `arduino_sensor_hub`
`maps/`, `tools/`	Sewer maps; world generator, dataset capture, training and evaluation scripts

Path	What is in it
`tests/`	27 test files, 479 tests (`fake_openrouter.py` is a local stand-in for the AI service)
`docs/`	`GOAL.md` , `PLANNER.md` , `BUILDER.md` , `LAB.md` , `HARDWARE.md` , `TRAINING.md` , `SETUP.md` , `LICENSING.md` , `RESULTS.md` , `RESULTS_LAB.md`

---

13. Glossary

- **Policy** – the thing that decides what a machine does (a hand-written controller, or one learned by RL).
- **Baseline** – the classical controller a learned policy must beat.
- **Reinforcement learning (RL)** – learning by trial and reward in many simulated runs.
- **Segmentation image** – a second render where every object has its own colour, giving exact labels.
- **Domain gap (sim-to-real)** – the difference between simulated and real images or physics, which makes simulator-trained models unreliable on real data.
- **Thrust-to-weight (TWR)** – total thrust divided by weight; a drone needs well over 1 to fly and about 2 to have reserve.
- **Brown-out** – a controller resetting because its supply voltage dips (the documented problem of an ESP32 on 4×AA cells without a boost converter).
- **Freeboard / draft** – height of the hull above / depth below the water.
- **Significant wave height (Hs)** – the usual measure of how rough the water is.
- **ODD (operating design domain)** – the conditions a machine is designed to operate in; stress testing explores its edges.
- **Fault injection** – deliberately making a part fail to see what the machine does.
- **Calibration** – measuring the real machine and correcting the model to match.
- **Confidence badge** – the label on every part's numbers: datasheet, typical, assumed or measured.

---

14. Where it goes next

1. **Fix and flash the rover**, then measure it (speed, spin, runtime) and calibrate its twin in the Builder: the first real check of the whole chain.
2. **A sharper camera** and fine-tuning the detector on real frames, to close the simulator-to-real gap.
3. **Position feedback** (wheel encoders or an IMU) so the real rover can follow a route by itself.
4. **A richer boat and drone model** (breaking waves, a fault-tolerant autopilot) and more goals in the catalogue.
5. **Prices and availability** in the parts library, and a free-text mode with an optional language model (kept optional so the tool still works offline).
6. **Real-hardware links for drones and boats**, built carefully, with the same interlocks as the rover.